PRACTICAL NO. 8

AIM: A system has RAM which can store four pages simultaneously to satisfy page requests. Write a program to calculate percentage page hit and page fault if the system uses Least recently used page replacement technique for the page references entered by the user i.e 1, 2,3,4,1,2,5,1,2,3,4,5 (No of Frames=3)

CODE:

MINGW64:/c/Users/airat/OneDrive/Desktop

```
  GNU nano 8.7                                                    lr
#include <stdio.h>

int main()
{
    int pages[] = {1,2,3,4,1,2,5,1,2,3,4,5};
    int n = 12;
    int frames[3];
    int time[3];
    int i, j;

    int hit = 0, fault = 0;

    // Initialize frames
    for(i = 0; i < 3; i++)
    {
        frames[i] = -1;
        time[i] = -1;
    }

    printf("LRU Page Replacement\n");
    printf("Number of Frames = 3\n\n");

    for(i = 0; i < n; i++)
    {
        int page = pages[i];
        int found = 0;

        // Check Hit
        for(j = 0; j < 3; j++)
        {
            if(frames[j] == page)
            {
                hit++;
                time[j] = i;
                found = 1;
                break;
            }
        }

        // If Fault → Replace LRU
        if(found == 0)
        {
            fault++;

            int lru_index = 0;

^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute       ^C
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/
```

MINGW64:/c/Users/airat/OneDrive/Desktop

GNU nano 8.7 lru.c

```
        // If Fault → Replace LRU
        if(found == 0)
        {
            fault++;

            int lru_index = 0;

            for(j = 1; j < 3; j++)
            {
                if(time[j] < time[lru_index])
                    lru_index = j;
            }

            frames[lru_index] = page;
            time[lru_index] = i;
        }

        // ---- OUTPUT FORMAT LIKE PDF ----
        printf("Page %d\n", page);

        for(j = 0; j < 3; j++)
        {
            if(frames[j] == -1)
                printf("Frame%d : -\n", j+1);
            else
                printf("Frame%d : %d\n", j+1, frames[j]);
        }

        if(found)
            printf("Result : Hit\n\n");
        else
            printf("Result : Fault\n\n");
    }

    float hit_per = (hit * 100.0) / n;
    float fault_per = (fault * 100.0) / n;

    printf("Total Hits = %d\n", hit);
    printf("Total Faults = %d\n", fault);
    printf("Hit Percentage = %.2f%%\n", hit_per);
    printf("Fault Percentage = %.2f%%\n", fault_per);

    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Und
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Red

OUTPUT :

```
Faiz@PUCKC3FO9N9DT MINGW64 ~/OneDrive/Desktop (master)
$ nano lru.c

Faiz@PUCKC3FO9N9DT MINGW64 ~/OneDrive/Desktop (master)
$ gcc lru.c -o lru

Faiz@PUCKC3FO9N9DT MINGW64 ~/OneDrive/Desktop (master)
$ ./lru
LRU Page Replacement
Number of Frames = 3

Page 1
Frame1 : 1
Frame2 : -
Frame3 : -
Result : Fault

Page 2
Frame1 : 1
Frame2 : 2
Frame3 : -
Result : Fault

Page 3
Frame1 : 1
Frame2 : 2
Frame3 : 3
Result : Fault
```

```
Page 4
Frame1 : 4
Frame2 : 2
Frame3 : 3
Result : Fault

Page 1
Frame1 : 4
Frame2 : 1
Frame3 : 3
Result : Fault

Page 2
Frame1 : 4
Frame2 : 1
Frame3 : 2
Result : Fault

Page 5
Frame1 : 5
Frame2 : 1
Frame3 : 2
Result : Fault

Page 1
Frame1 : 5
Frame2 : 1
Frame3 : 2
Result : Hit
```

```
Page 2
Frame1 : 5
Frame2 : 1
Frame3 : 2
Result : Hit

Page 3
Frame1 : 3
Frame2 : 1
Frame3 : 2
Result : Fault

Page 4
Frame1 : 3
Frame2 : 4
Frame3 : 2
Result : Fault

Page 5
Frame1 : 3
Frame2 : 4
Frame3 : 5
Result : Fault

Total Hits = 2
Total Faults = 10
Hit Percentage = 16.67%
Fault Percentage = 83.33%
```